Answer Box #1

For the records that implement Region, which are base cases of the data and which are recursive cases

RectRegion: Base
CircRegion: Base
Union: Recursive
Intersect: Recursive
Negate: Recursion

```
record Union(Region l, Region r) implements Region
    boolean contains(Point p){
        return l.contains(p) || r.contains(p);
    }
```

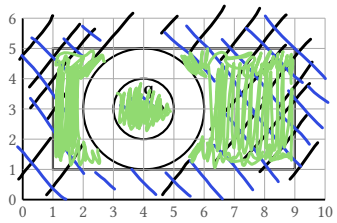
Region Calls

Name: _____ Date: _____

```
record Point(double x, double y) {}
interface Region { boolean contains(Point p); }
record RectRegion(Point corner, double width, double height) implements Region { ... }  // as last time
record CircRegion(Point center, double radius) implements Region { ... }                // as last time
record Union(Region a, Region b) implements Region {
  public boolean contains(Point p) { return this.a.contains(p) || this.b.contains(p); } }
record Intersect(Region a, Region b) implements Region {
  public boolean contains(Point p) { return this.a.contains(p) && this.b.contains(p); } }
record Negate(Region r) implements Region {
  public boolean contains(Point p) { return !this.r.contains(p); } }
```

```
Point corner = new Point(1, 1);
Region card = new RectRegion(corner, 8, 4);
Point mid = new Point(4, 3);
Region hole = new CircRegion(mid, 2);
Region plug = new CircRegion(mid, 1);
Region outside = new Negate(hole);
Region either = new Union(outside, plug);
Region shape = new Intersect(card, either);
Point q = new Point(4.5, 3);
boolean in = shape.contains(q);
```

Q1. Shade `shape`.



/// outside
\\\ after union with plug
shape

Q2. Write `shape` as one expression, using only `card`, `hole` and `plug`.

_____

Q3. Fill in memory just before `shape.contains(q)` is called.

NAMES (THE STACK)

top level
corner = @1001
card = @1002
mid = @1003
hole = @1004
plug = @1005
outside = @1006
either = @1007
shape = @1008
q = @1009

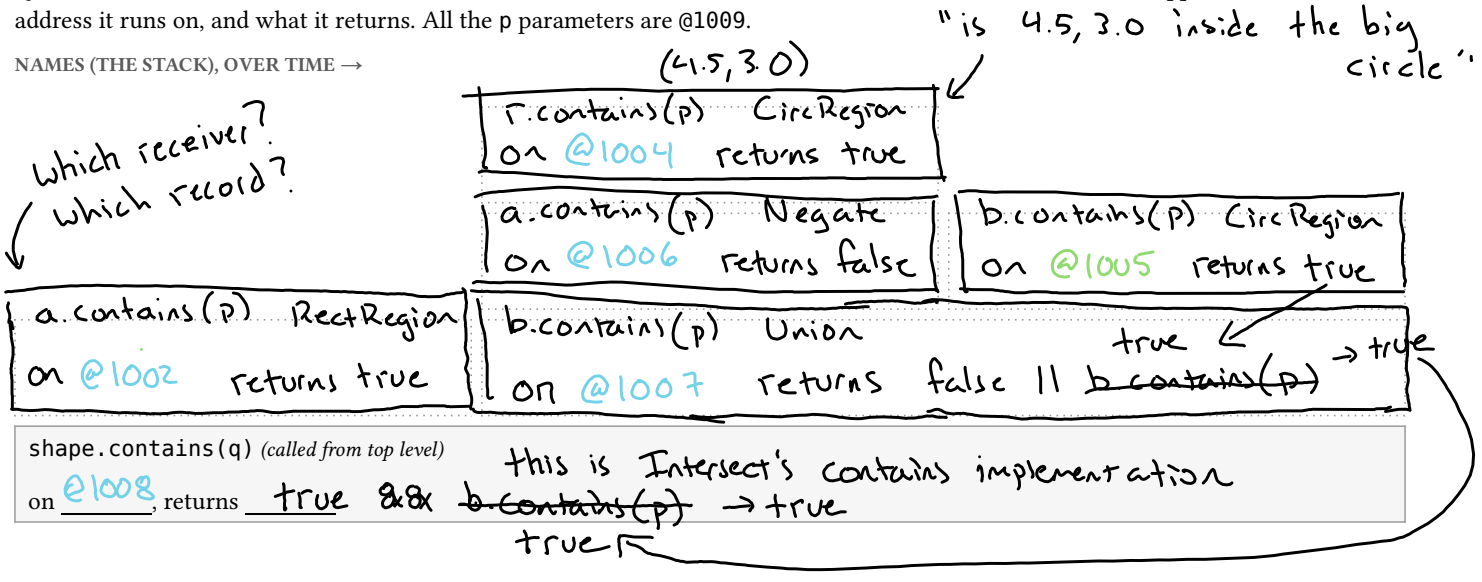
HEAP

@1001: Point[x=1.0, y=1.0]
@1002: RectRegion[corner=@1001, width=8.0, height=4.0]
@1003: Point[x=4.0, y=3.0]
@1004: CircRegion[center=@1003, radius=2.0]
@1005: CircRegion[center=@1003, radius=1.0]
@1006: Negate[r=@1004]
@1007: Union[a=@1006, b=@1005]
@1008: Intersect[a=@1002, b=@1007]
@1009: Point[x=4.5, y=3.0]

/// outside
\\\ union
final shape

Q4. Draw the calls of `shape.contains(q)` over time. Each box is one call of `contains`. Write the call as it appears in the code, the address it runs on, and what it returns. All the `p` parameters are `@1009`.

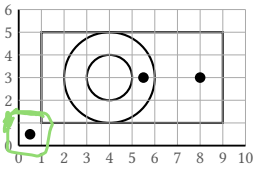
NAMES (THE STACK), OVER TIME →



answer box #2

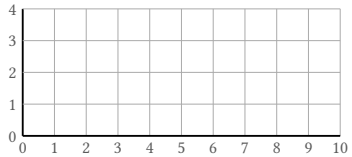
Q5. For each point below, how many calls of `contains` run, and what does `shape.contains` answer?

point	calls	answer	why it stops there
(0.5, 0.5)	2	false	false && b.contains(p)
(8, 3)			
(5.5, 3)			




```
Point c0 = new Point(1, 2);
Region body = new CircRegion(c0, 1);
for (int i = 1; i < 4; i = i + 1) {
  Point c = new Point(1 + 2 * i, 2);
  Region seg = new CircRegion(c, 1);
  body = new Union(body, seg);
}
// ★
Point tip = new Point(7, 2);
boolean hit = body.contains(tip);
```

Q6. Shade `body`.



Q7. Fill in memory at ★, after the loop. `i`, `c` and `seg` belong to the loop and are gone; cross out the old values of `body`.

NAMES (THE STACK)

top level
`c0` = `@1001`
`body` = _____

HEAP

`@1001`: `Point[x=1.0, y=2.0]`
`@1002`: `CircRegion[center=@1001, radius=1.0]`
`@1003`: `Point[x=3.0, y=2.0]`
`@1004`: `CircRegion[center=@1003, radius=1.0]`
`@1005`: _____
`@1006`: _____
`@1007`: _____
`@1008`: _____
`@1009`: _____
`@1010`: _____
`@1011`: _____

Q8. Draw the calls of `body.contains(tip)` over time, the same way. All the `p` parameters are `@1012`.

NAMES (THE STACK), OVER TIME →

`body.contains(tip)`
on _____, returns _____

top level
`c0`, `body` as in Q7, `tip` = `@1012`, `hit` = _____

Q9. Which circle was built first? _____ Which circle does `body.contains` check first? _____ Where is that call on the stack: the deepest or the shallowest? _____ Explain why the first one built is the first one checked.